

实验三——图像分类及经典 CNN 实现

10211900416 郭夏辉 2023 年 11 月

实验目的：使用多种 CNN 模型进行 MNIST 手写数字识别

实验要求：使用至少 4 种 CNN 架构，其中必须包含 LeNet, AlexNet, ResNet

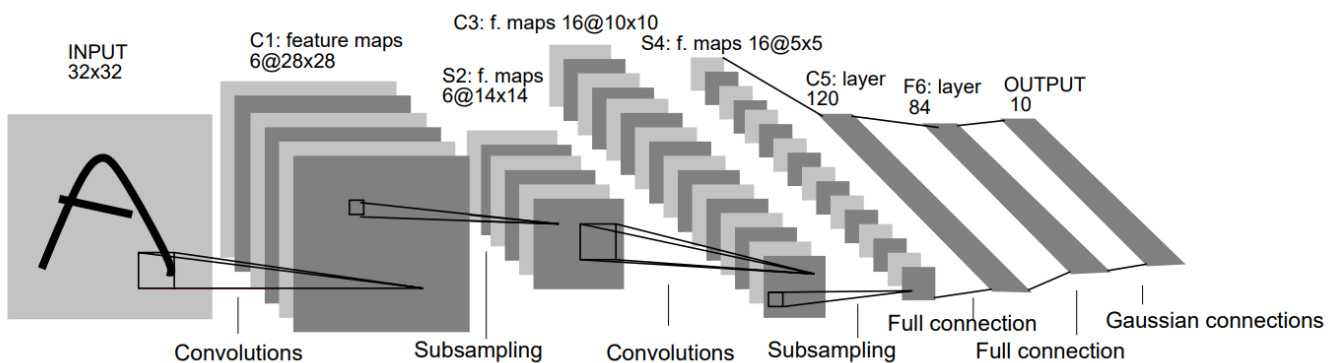
环境配置：Python 3.10.9+Pytorch

实验过程

我在本次实验中，不仅实现了要求的三种 CNN 架构（LeNet, AlexNet, ResNet），而且还额外地实现了三种架构——VGGNet, MobileNet 和 GoogLeNet。现在我先谈一下这几种模型的架构和我是怎么设计的。

LeNet

LeNet 是最早发布的 CNN 之一，在 1998 年被 Yann LeCun 等人所提出，他们通过卷积、池化、全连接等方法完成了特征抽取这一工作，具有很强的划时代性。虽然 LeNet 有很多的变种，本次实验还是实现了最原始的论文（Gradient-based learning applied to document recognition, 1998.）中提出的 LeNet-5 网络：



整个 LeNet-5 网络共 7 层：

C1 卷积层：卷积核大小 5，步长 1，卷积核数量 6，输入 $1 \times 32 \times 32$ ，输出 $6 \times 28 \times 28$ ；

S2 池化层：卷积核大小 2，步长 2，输入 $6 \times 28 \times 28$ ，输出 $6 \times 14 \times 14$ ；

C3 卷积层：卷积核大小 5，步长 1，卷积核数量 16，输入 $6 \times 14 \times 14$ ，输出 $16 \times 10 \times 10$ ；

S4 池化层：卷积核大小 2，步长 2，输入 $16 \times 10 \times 10$ ，输出 $16 \times 5 \times 5$ ；

C5 卷积层：卷积核大小 5，步长 1，卷积核数量 120，输入 $16 \times 5 \times 5$ ，输出 $120 \times 1 \times 1$ ；

F6 全连接层：输入：120 维，输出 84 维；

OUTPUT 全连接层：输入：84 维，输出 10 维

有一点值得注意，在原始的论文中，LeNet 中的池化层进行的是平均池化而不是最大池化，使用的激活函数还是 Sigmoid。为了更加优化，我在本次实验中 LeNet 使用的激活函数为 ReLU。

LeNet-5 在当时的手写数字识别任务中取得了很好的效果，可以达到 98% 以上的准确率，这是当时最先进的技术水平，也为后来更加复杂的卷积神经网络奠定了基础。

AlexNet

虽然 LeNet 在小数据集上取得了很好的效果，但是在更大、更真实的数据集上训练卷积神经网络的性能和可行性还有待研究，因此卷积神经网络在计算机视觉和机器学习领域还称不上主导地位。在 Imagenet classification with deep convolutional neural networks, 2012. 中，Alex Krizhevsky 等人提出的 AlexNet 架构夺得了 2012 年 ImageNet ILSVRC 比赛的冠军，首次证明了学习到的特征可以超越手工设计的特征，开启了神经网络图识的新纪元。

AlexNet 是较为庞大的，其作者将其分为了两个部分——特征提取层和全连接层。AlexNet 之所

以这么优秀，离不开它的特性。它在全连接层使用了 dropout 来减轻过拟合现象（dropout 的原理此处不展开，理论课已经有所介绍了，简而言之就是随机 cut 掉一些隐藏层的结点，这样每次传播时参数就仅为网络的一部分，降低了模型的过拟合程度）；它使用的激活函数是 RELU 减少计算量的同时有效缓解了梯度消失现象；它的深度更深，将图片不同的特征反映到 feature_map 中，对于特征学习的效果更好。在原始论文中，AlexNet 采用 GPU 而不是 CPU 进行模型的训练，有效地提升了模型训练的效率。AlexNet 和 LeNet 的架构非常相似，整个 AlexNet 网络共 11 层：

卷积层 1：卷积核大小 11，步长 4，卷积核数量 96，输入 $1 \times 227 \times 227$ ，输出 $96 \times 55 \times 55$

池化层 1：最大池化，卷积核大小 3，步长 2，输入 $96 \times 55 \times 55$ ，输出 $96 \times 27 \times 27$

卷积层 2：卷积核大小 5，步长 1，卷积核数量 256，填充 2，输入 $96 \times 27 \times 27$ ，输出 $256 \times 27 \times 27$

池化层 2：最大池化，卷积核大小 3，步长 2，输入 $256 \times 27 \times 27$ ，输出 $256 \times 13 \times 13$

卷积层 3：卷积核大小 3，步长 1，卷积核数量 384，填充 1，输入 $256 \times 13 \times 13$ ，输出 $384 \times 13 \times 13$

卷积层 4：卷积核大小 3，步长 1，卷积核数量 384，填充 1，输入 $384 \times 13 \times 13$ ，输出 $384 \times 13 \times 13$

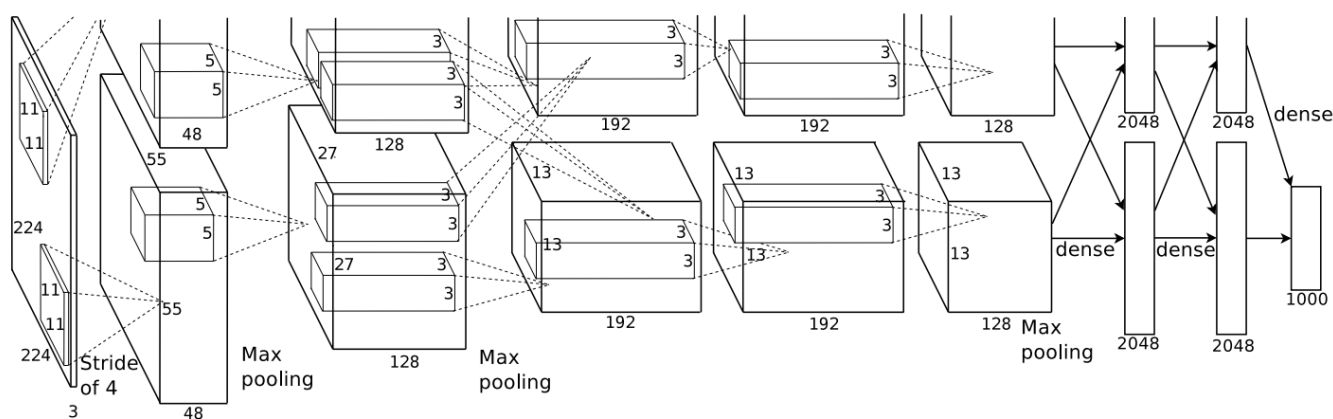
卷积层 5：卷积核大小 3，步长 1，卷积核数量 256，填充 1，输入 $384 \times 13 \times 13$ ，输出 $256 \times 13 \times 13$

池化层 3：最大池化，卷积核大小 3，步长 2，输入 $256 \times 13 \times 13$ ，输出 $256 \times 6 \times 6$

全连接层 1：输入： $256 \times 6 \times 6 = 9216$ 维，输出 4096 维

全连接层 2：输入：4096 维，输出 4096 维

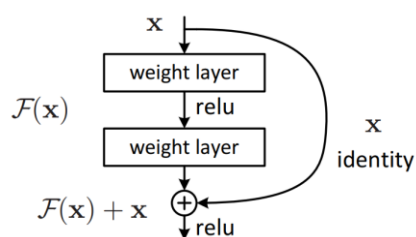
全连接层 3：输入：4096 维，输出 10 维



ResNet

在 Deep residual learning for image recognition, 2016. 中，Kaiming He 等人通过残差连接这一方法，解决了网络层数加深导致的特征消失问题，并实现了最深 152 层的神经网络。

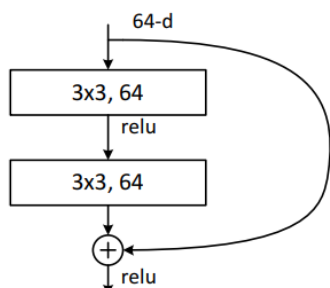
在神经网络达到一定的深度后，很容易出现梯度爆炸/消失现象，而且这种现象随着深度的增大愈发明显，这也是制约传统的 CNN 层数的根本原因，但是如果想让模型得到更全面的特征信息，我们就必须加深网络层数。ResNet 中利用 batch normalization 加速训练，不再用 dropout（全局平均池化层足以代替多层全连接层）的同时，提出了 Shortcut Connection，利用残差神经网络，使得神经网络的深度首次突破了 100 层，甚至之后的变种可以达到 1000 层。



ResNet 中的 Shortcut Connection 思想如左图所示，这也是一个残差块的基本结构。梯度有一部分直接传递到了下一层，另一部分才是通过 F 函数传递的，而不是像传统的 CNN 那样直接将上一层的输出结果作为下一层的输入，所以整体来说这个梯度几乎不会为 0，网络的性能不易退化，同时也能避免梯度无效的情况。

根据 ResNet 原始的论文，它有两种残差块——BasicBlock 和 Bottleneck。前者主要应用在浅层残差网络（18, 34），后者主要用于深层残差网络（50 层以上）。

本次实验中我实现的 ResNet 为 18 层，所以使用的残差块是 BasicBlock，具体如左图所示。BasicBlock 包含两个卷积层，输入经过两个卷积层后得到输出与输入相加，再经过激活函数然后输出到下一层。



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10 ⁹	3.6×10 ⁹	3.8×10 ⁹	7.6×10 ⁹	11.3×10 ⁹

其实，ResNet 我还有一些额外的话想要补充。ResNet 最开始两层和我之后要实现的 GoogLeNet 一样，在输出通道数为 64，卷积核大小 7，步长 2 的卷积层之后是卷积核 3，步长为 3 的最大池化层，但之后不同就展现了——ResNet 在每个卷积层后增加了批量规范化层；GoogLeNet 在后面接了 4 个由 Inception 块组成的模块，ResNet 则使用 4 个由残差块组成的模块。

VGGNet

在论文 Very Deep Convolutional Networks for Large-Scale Image Recognition, 2015 中，Visual Geometry Group 证明了增加网络的深度能够在一定程度上影响网络最终的性能，提出了 VGGNet 这一架构。VGGNet 其实和之前讲到的 AlexNet 有很多相似性，但是也有很大的提升。其中最令我印象深刻的是，VGGNet 采用连续的几个 3x3 的卷积核代替 AlexNet 中的较大卷积核（11x11，7x7，5x5），

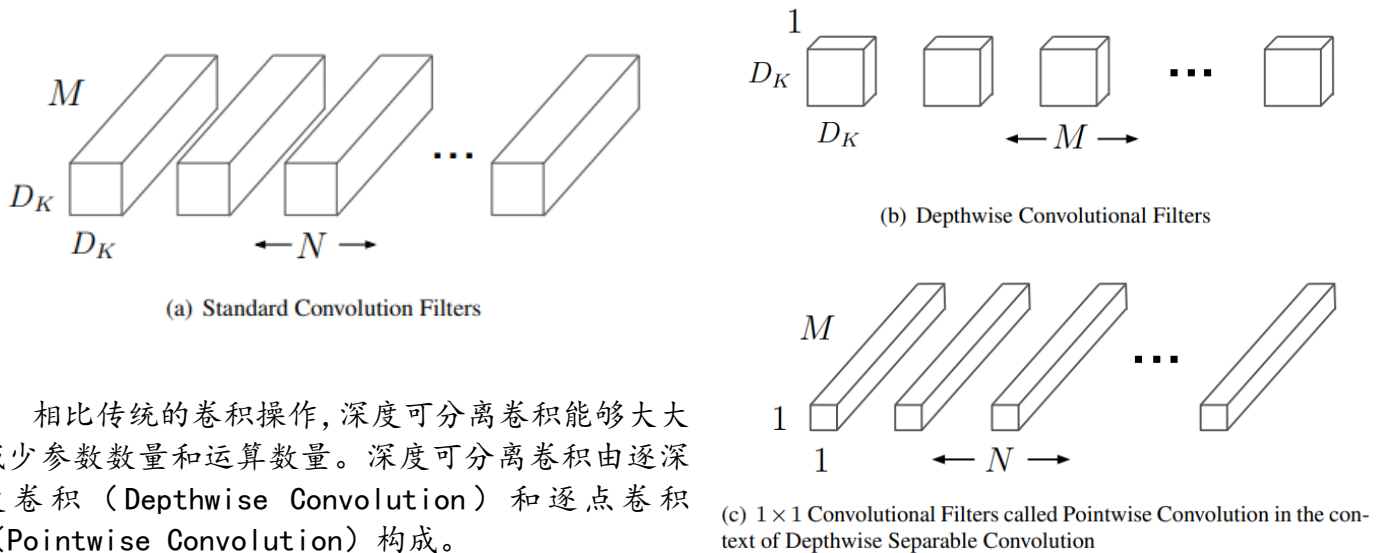
这样就对于相同的感受野来说，通过一次次的卷积，慢慢地缩小特征响应图。采用堆积的小卷积核优于采用大的卷积核，因为多层非线性层可以增加网络深度来保证学习更复杂的模式，而且代价还比较小（参数更少）。因此，VGGNet 可以比 AlexNet 有更深的网络深度。也类似于我在之前介绍的 ResNet，采用的是小卷积核而不像是早期 CNN 那样的大卷积核。当然见识过 ResNet 威力的我们也清楚，这个“治标不治本”的策略还是无法解决很深层次 CNN 时遇到的梯度爆炸/消失问题。

VGGNet 论文中介绍了 6 种不同结构的网络。我选取的 VGGNet 结构是 D 类（16 层）。在代码的实现上为了使整个模型更加灵活，我实现了一个 VGGBlock 结构，它可以调整输入/输出通道数以及网络深度。因为 VGGNet 中很多层有着相似性，也就过若干层之间的池化层需要对特定数据调整。而且为了提高效率，我将输入图片尺寸修改为 64x64，并且对应地也修改了全连接层结构。

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

MobileNet

在 MobileNets, Efficient Convolutional Neural Networks for Mobile Vision Applications, 2017 中, Andrew Howard 等人提出了基于深度可分离卷积 (Depthwise Separable Convolution) 的 MobileNet 架构。这一轻量级的架构可能准确性并没有前面几种架构那么优秀, 但是对计算资源的消耗是比较少的, 使得在移动端进行训练成为了现实。从我个人的角度来说, MobileNetV1 就是把 VGGNet 中的标准卷积层换成深度可分离卷积, 它的核心也在于深度可分离卷积, 我来围绕它主要介绍一下。



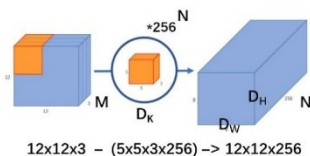
相比传统的卷积操作, 深度可分离卷积能够大大减少参数数量和运算数量。深度可分离卷积由逐深度卷积 (Depthwise Convolution) 和逐点卷积 (Pointwise Convolution) 构成。

在输入图片大小 $D_K \times D_K \times M$ 为时, 传统的卷积操作计算量为 $D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F$

但是深度可分离卷积的计算量仅为 $D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F$

知乎这篇文章上的这张图 (<https://zhuanlan.zhihu.com/p/70703846>) 也很好地诠释了为何深度可分离卷积能够减少参数数量和运算数量:

深度可分离卷积 标准卷积参数与计算量



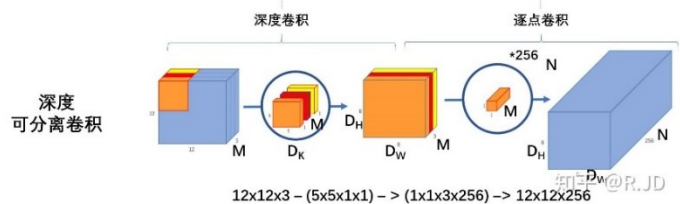
$$\text{Params} \quad D_K \times D_K \times M \times N$$

$$\text{计算量} \quad D_K \times D_K \times M \times N \times D_W \times D_H$$

深度可分离卷积 DSC卷积参数与计算量

$$\text{Params} \quad D_K \times D_K \times M + M \times N$$

$$\text{计算量} \quad D_K \times D_K \times M \times D_W \times D_H + M \times N \times D_W \times D_H$$



概括而言, 逐深度卷积 (Depthwise Convolution) 针对每个输入通道采用不同的卷积核, 不像传统卷积运算那样卷积核用在所有的输入通道, 逐深度卷积每个输入通道由此可以对应不同的卷积核; 逐点卷积 (Pointwise Convolution) 本质来说就是一个卷积核大小为 1 的传统卷积。

有一点需要注意, 就是 MobileNet 中使用的激活函数不是 ReLU, 而是 ReLU6。ReLU6 作为非线性激活函数, 在低精度计算下具有更强的鲁棒性, 这一激活函数将网络的输出最大结果限制为 6。使用这个激活函数, 也可以防止精度缺失导致的网络不正常收敛问题。而且 MobileNet 减少参数数量和计算成本情况下, 不依赖 dropout 进行正则化, 但是 MobileNetV2 中在训练期间却使用了 dropout。

我在此实验中所采用的是 MobileNetV1 架构, 它的结构如下图所示。可以看到, 与 VGGNet 类似, MobileNet 同样使用了连续的小卷积块来对不同尺度的 feature 进行抽取。其卷积层通道数从 32 出发, 经历 64, 128, 256, 512, 1024 最终升高到较大的输出通道。

类似于自己之前的代码实现思想, 我在实现 MobileNet 时先实现了那个深度可分离卷积模块 MobileBlock, 在此基础上根据左图的架构进行实现。

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5× Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

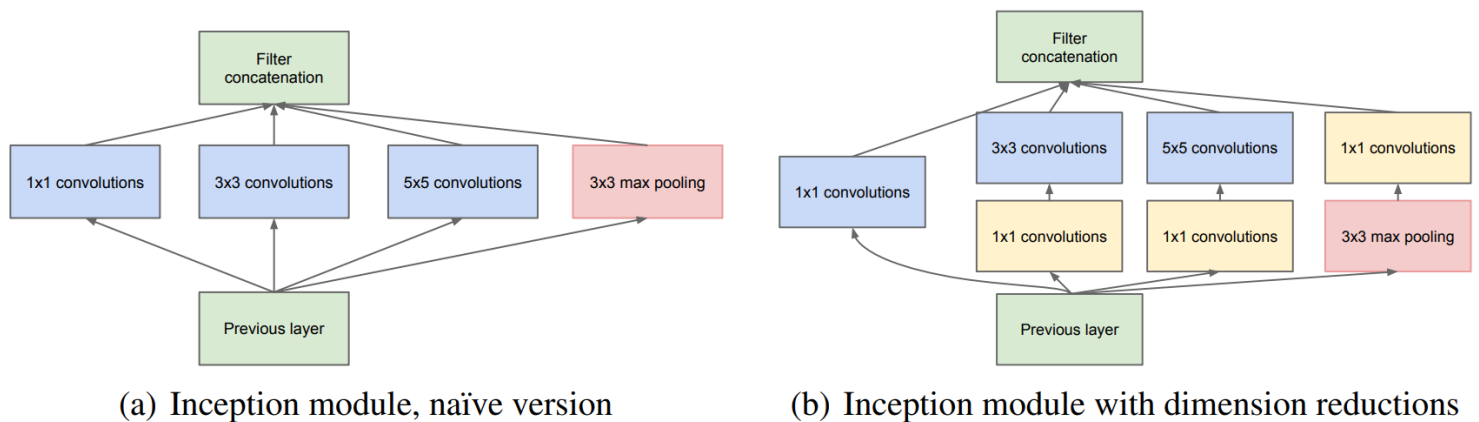
为了进一步提升 MobileNet 的运行效率，我并没有采用原始的 224×224 的图片作为输入，转而将原始图片缩放到 128×128 进行输入。

GoogLeNet

GoogLeNet 是一种较为复杂的结构，在 Going Deeper with Convolutions, 2014 中由 Google 的 Christian Szegedy 等人提出。VGGNet 继承了 LeNet 以及 AlexNet 的一些框架结构，而 GoogLeNet 则做了更加大胆的网络结构尝试，虽然深度只有 22 层，但大小却比 AlexNet 和 VGGNet 小很多。

一般来说，提升网络性能最直接的办法就是增加网络深度和宽度，深度指网络层数数量、宽度指神经元数量。但这种方式存在以下问题：1. 参数太多，如果训练数据集有限，很容易产生过拟合；2. 网络越大、参数越多，计算复杂度越大，难以应用；3. 网络越深，容易出现梯度弥散问题（梯度越往后穿越容易消失），难以优化模型。解决这些问题的方法当然就是在增加网络深度和宽度的同时减少参数，为了减少参数，自然就想到将全连接变成稀疏连接。但是在实现上，全连接变成稀疏连接后实际计算量并不会会有质的提升，因为大部分硬件是针对密集矩阵计算优化的，稀疏矩阵虽然数据量少，但是计算所消耗的时间却很难减少。

GoogLeNet 提出了 Inception 结构，融合不同尺寸的特征信息，它的结构如下所示：



左图是论文中提出的 Inception 原始结构，右图是 Inception 加上降维功能的结构。左图中 Inception 结构一共有 4 个分支，也就是说我们的输入的特征矩阵并行的通过这四个分支得到四个输出，然后在将这四个输出在深度维度进行拼接得到最终输出（为了让四个分支的输出能够在深度方向进行拼接，必须保证它们输出的特征矩阵高度和宽度都相同）。右图对比左图，就是在分支 2, 3, 4 上加入了卷积核大小为 1×1 的卷积层进行降维以及映射处理，减少模型训练参数，过略冗余参数，进而减少计算量。

它还添加两个辅助分类器帮助训练，为了防止梯度无法回传导致浅层神经网络无法训练，辅助分类器用于向前传导梯度。反向传播时如果有一层求导为 0，链式求导结果则为 0。而且辅助分类器将中间某一层输出用作分类，起到模型融合作用。

它丢弃全连接层，使用平均池化层，进一步减少参数。由于全连接网络参数多，计算量大，容易过拟合，所以 GoogLeNet 没有采用 VGGNet、LeNet、AlexNet 三层全连接结构，直接在 Inception 模块之后使用 Average Pool 和 Dropout 方法，不仅起到降维作用，还在一定程度上防止过拟合。

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

我在实际的实验过程中实现的是最原始的 GoogLeNet，采用的还是 Inception V1，而且为了提速输入图片尺寸是 128×128，之后的 Google 开发人员继续研发，有了升级版本的 GoogLeNet。

遇到的主要问题与解决

1. 数据的维度问题

我们的数据集是经典的 MNIST 数据集，它的训练集是 60000 张图像，测试集是 10000 张图像，这些数据的标签都是给出的。这些图像都是单通道的黑白图像，尺寸皆为 28×28。然而，我实现的六种模型原始论文中尺寸都不是这样的——LeNet 是 32×32，其他的原始论文中都是 224×224。因此，我要经过一个转化的过程，使得输入数据和我们的网络“对齐”。

为了便利图片的放缩，我采用的是 torchvision.transforms 的 Resize 方法，在 Resize 之后再 train。当然还可能出现通道数不一致的问题，因为 Resize 并不能改变数据的通道数，比如 AlexNet 中原始论文的输入图片是 3 通道 224×224 的，对于这个问题我们只需要更改对应的 in_channels=1 便能解决。在每个 epoch 中，将 x 用 torch_resize 改变图片维度，一个一个地将各 batch 改变维度，而不是一股脑把所有的 70000 张图片都改了维度，这样也能解决内存溢出的问题。

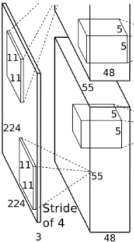
2. 数据的输入问题

考虑到网络的不稳定性，我没有使用 tensorflow.keras.datasets 中的 mnist 来导入数据，而是采用下载官方的数据集之方法，但是对于解压得到的二进制文件我要读取还要遵循一些格式。

经过上网查询，我在打开二进制文件后，要解析数据的魔数、个数、宽高等信息，在此之后才能解析像素值，并进行张量化操作。¹²在张量化之后，我使用 torch.utils.data 的 DataLoader 功能对数据集进行了分组，使用 sklearn.model_selection 的 train_test_split 方法对训练集和验证集进行了划分（划分比例是训练集:验证集=4:1），为之后实验的展开奠定了基础。

3. AlexNet 论文中网络的输入维度和实际不一致

在 AlexNet 的官方论文中，输入层的情况应该是 3×224×224，卷积核大小为 11，数量 96，步长是 4，输出是 96×55×55。当然我输入的图片宽度是单通道的，那么这个图片的尺寸 224×224 真的是正确的吗？在理论课上，老师教过我们原图尺寸是 N，卷积核的大小是 F，步长是 stride，填充长度为 padding，那么输出的宽度是向下取整 $(N+2*padding-F)/stride+1$ 。但是如果按照原始论文，结果应该是 $(224+2*0-11)/4+1=54$ ，并不能与原论文的 55 输出尺寸对应起来。因此，这里我采用了 1×227×227，padding=0 的图片作为输入来和 55 的输出尺寸对应起来。



4. 显存/内存爆炸问题

在实验最开始的阶段我遇到了内存迅速被填满的情况，这让我很不可思议——数据集也并没有很大，自己 4G 的显存+16G 的内存完全是够用的啊。后来才发现我在 train 的过程中反复地调用 get_acc 获取准确率，但是自己忘记 with torch.no_grad() 来停止对梯度的计算和存储，从而减少对内存的

¹ https://blog.csdn.net/qq_21792169/article/details/103324368
² <https://blog.csdn.net/hxxjxw/article/details/113727973>

消耗，进行了大量没有意义的反向传播。加上这个语句之后，问题立马就解决了。

5. 统一的训练方法

本文我实现了 6 种 CNN 架构，它们其实有着相似的训练过程，都是采用反向传播算法来训练的，通过最小化损失函数来优化网络的权重和偏置。为了减轻自己代码的实现难度，我把训练的相应功能包装到了 `tools/train.py` 中。损失函数我都采用的是交叉熵作为损失函数，优化器都采用的是相对来说更好一些的 Adam，并在此过程中加入了基于不同设备（cuda 还是 cpu）的对应过程。

但是，正常的采用辅助分类函数的 GoogLeNet 架构输出较为特殊，它返回的是多个数值，但是只有一个是我们需要的。为了不破坏代码的模块性，我痛定思痛，虽然自己已经实现了相应的功能，但是我还是决定不使用 AuxClassifier。

6. 围绕图片尺寸，训练速度和质量的取舍

类比遥感图像和日常拍摄的图像，不同的数据集输入尺寸其实是对模型的训练有所影响的。图片放缩地越大，显然其纹理和上下文信息越多，进行特征抽取工作的表现应该更好。但是经过实测，对于同样的模型，扩大其输入尺寸，真的像除 LeNet 那样遵循原始论文以 224×224 输入图片，AlexNet 和 ResNet 的时间很庞大，但能接受；但是 VGGNet, GoogLeNet 的训练时间长到无法接受，我在跑 VGGNet 时甚至花了一整个晚上才跑出来一个 epoch，GoogLeNet 甚至一下午一个 epoch 都没有好。为了更好地展现模型，我选择缩小输入尺寸（当然也会对应地修改全连接层的结构）、适当降低训练的准确性。

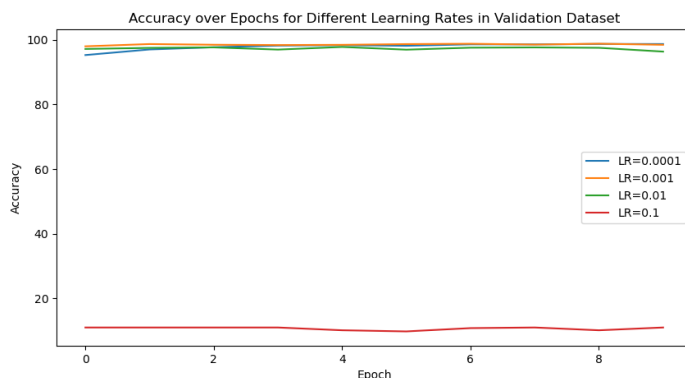
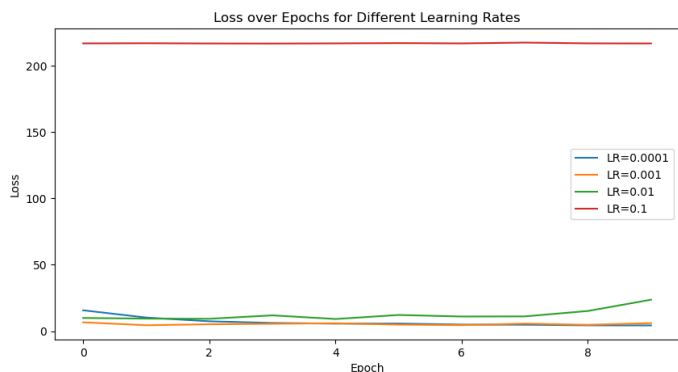
7. 围绕 batch，训练速度和质量的取舍

使用更大的 batch 在提高运算速度与训练稳定性的同时，但是也会带来显存不足、模型泛化能力差等问题。本来我选择的 batch_size 为 128，但经过实测，VGGNet 时会出现 cuda out of memory 显存不足错误，所以此时要特别考虑。

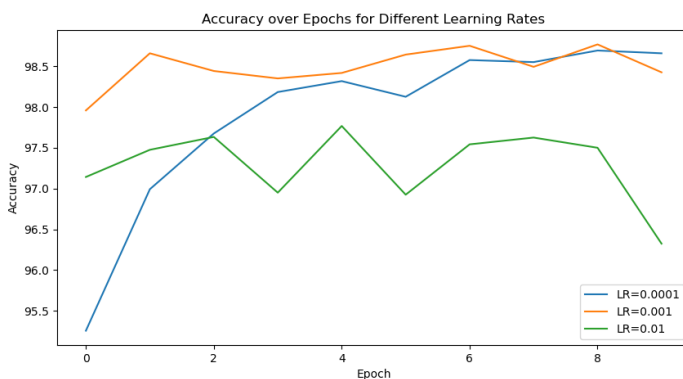
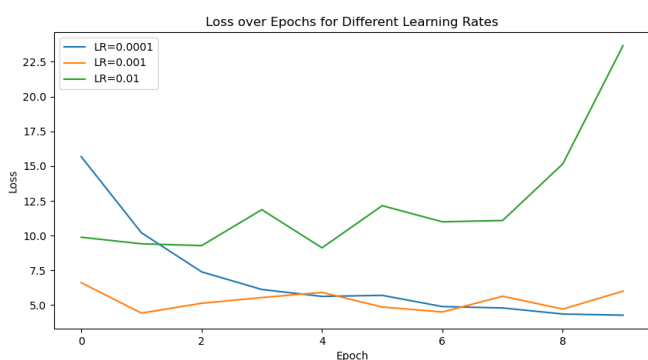
模型之间的对比及分析

学习率我应该取多少？

为了更好地对比各个模型，某些超参数的选择 6 种模型应该是一致的，像第一个实验那样每种模型都网格搜索出一个“最好的”超参数虽然有一些研究意义，但实际并无必要。以 LeNet 为例，我来展现一下学习率对训练的影响吧。（此时 epoch 为 10，batch_size 为 128）

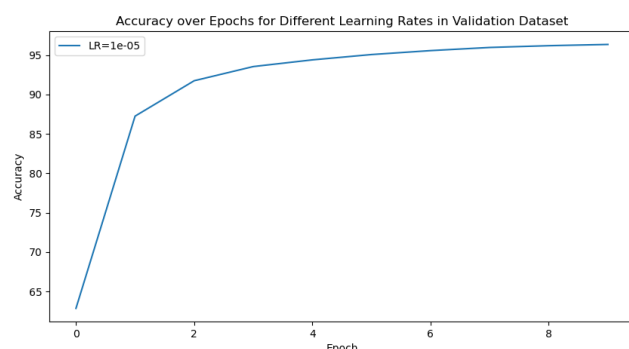
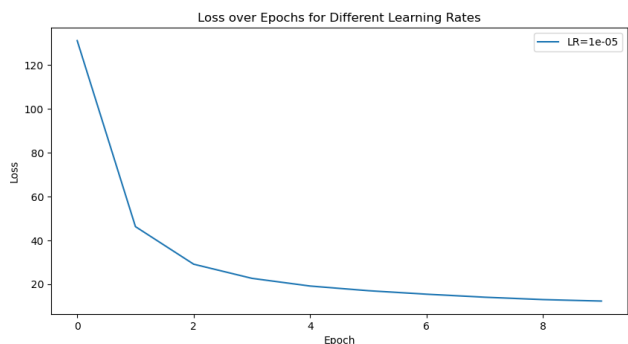


除去学习率为 0.1 再来更加直观地看：



可以看到，学习率较大（0.1）时，随着 epoch 的增大，无论是损失函数还是在验证集上的准确率都近乎一条直线，而且情况都是稳定的“糟糕”。这个其实也不难理解，学习率控制模型参数在每

一次更新中的调整幅度，学习率较大时，损失函数可能直接就超过了全局最优点，易发生梯度爆炸情况，而且我们可以很明显地看到学习率为 0.01 时地表现不如学习率为 $1e-4$ 和 $1e-3$ 。学习率如果较小情况下，我们也知道损失函数变化很慢，易于过拟合。在本次实验中，我测试到如果学习率为 $1e-5$ ，效果虽然可以接受，但已经不太妙了：



因此，经过一些取舍，我在 $1e-5$ 和 $1e-1$ 中间的数量级选择了 $1e-4$ 作为默认的学习率。而且自己采用的优化器是 Adam，本身就有自适应学习率算法，在训练过程中动态地调整学习率，所以我觉得自己选择的 $1e-4$ 是比较折中的。

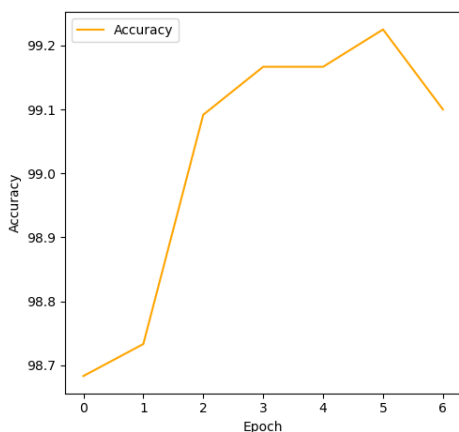
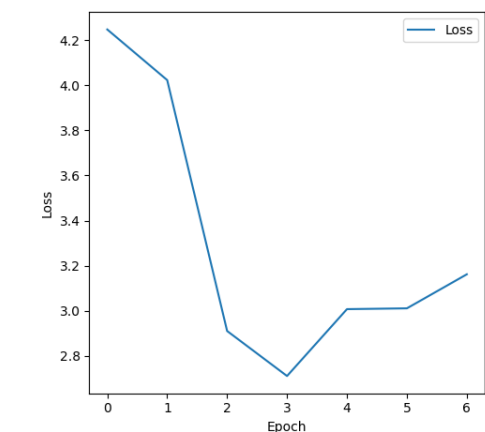
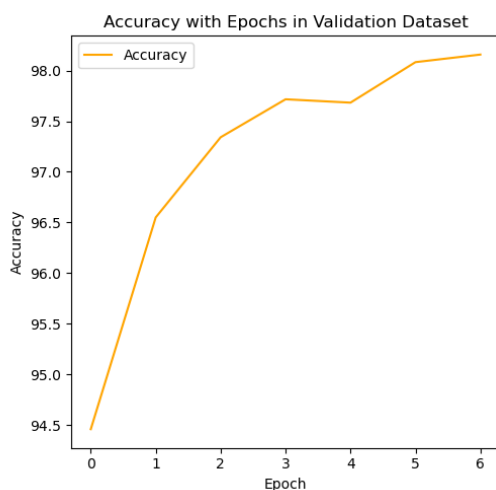
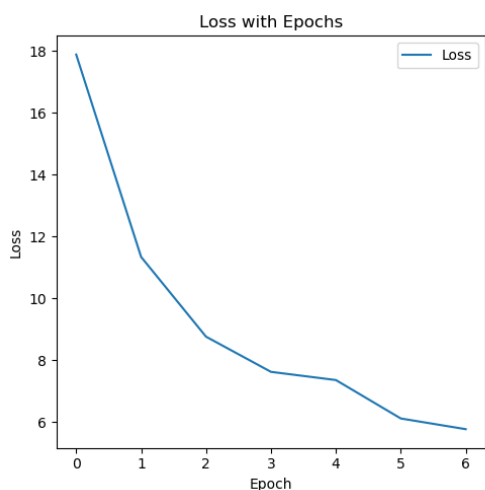
dropout 的值我应该如何选择？

除了学习率，模型还有一个重要的超参数便是 dropout，这个的选择也是有所讲究的。一方面，有的模型并没有使用 dropout 这个机制，所以不用去调整——LeNet, ResNet, MobileNet。有的架构，比如 GoogLeNet，虽然把全连接替换成了全局均值池化，但这后面还是使用了 0.4 的 dropout，这个 0.4 是我查阅众多资料后发现的 Inception V1 采用的经典 dropout 数值，我无需调整。因此我主要需要考虑 dropout 的模型就两种——AlexNet 和 VGGNet。

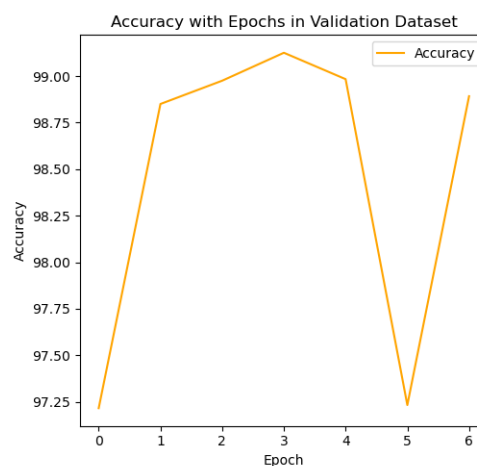
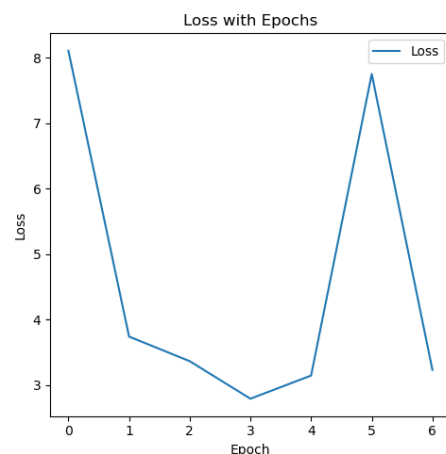
过低的 dropout 可能导致过拟合，而过高的 dropout 可能导致模型收敛缓慢或者在训练过程中信息丢失过多，因此实践中一般就事论事地去设计 dropout 率。为了尽可能地折中，我采用的 dropout 统一为 0.5。

各模型结果

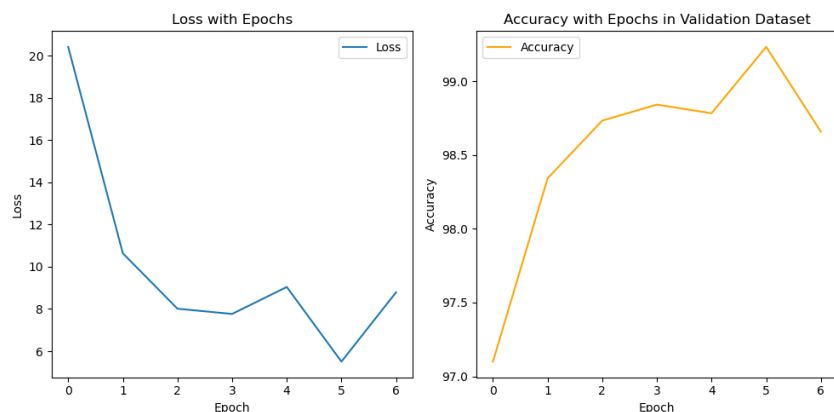
我每个模型都跑了 7 个 epoch，每个模型的训练时损失函数，验证集准确率情况如下所示：
上 LeNet 下 AlexNet：



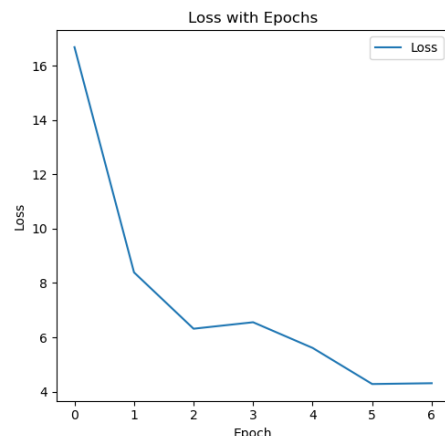
ResNet：



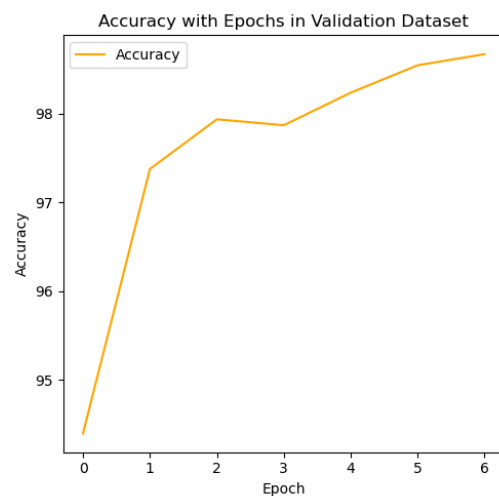
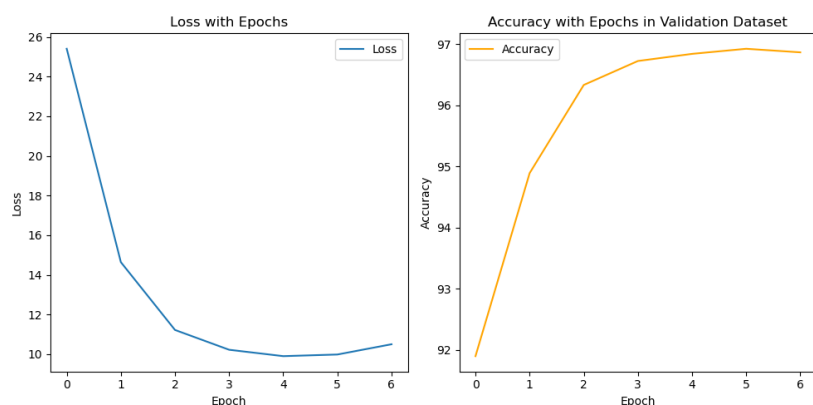
VGGNet:



GoogLeNet:



MobileNet:



	LeNet	AlexNet	ResNet	VGGNet	MobileNet	GoogLeNet
训练时间/s	162.601	900.754	2358.581	1079.028	1063.712	1117.1243
测试集上的准确率/%	98.050	99.231	99.303	98.751	97.162	98.598
测试集上的损失函数值	4.464	1.946	2.631	6.767	7.979	3.3673
输入图片的尺寸	32×32	227×227	224×224	64×64	128×128	128×128
深度(包括全连接层)	7	11	18	16	30*	22
参数数量	61,706	58,299,082	11,175,370	15,306,442	3,216,682	5,977,530

*指的是包括全连接层和平均池化层，并且把深度卷积和逐点卷积分开算

我对结果的理解

有关模型的细节我在此不展开描述了，前文已经花了很多篇幅在阐述，这里我主要写一些自己的感受。

首先，在实验过程中，我能明显感受到随着输入图片缩放尺寸之扩大，模型的效果更好了 (AlexNet 和 ResNet)，但是图像过大会导致模型复杂性提高，所耗费的训练时间也明显增长，这也

是我之后实现的三种模型都没有采用原始论文中那样的 224×224 尺寸之核心原因。以 VGGNet 为例，如果输入的图像尺寸是 128×128 (也会对应地修改全连接层的结构)，则模型参数数量为 24,163,018，但与之相对，如果采用 64×64 的图像，模型参数数量仅为 15,306,442，在损失不了多少准确率的同时，可以极大地减少训练耗时。参数数量提高，意味着训练时间的增长，也似乎带来了学习效果的提升。但是这个关联并不是确切的，因为我并没有去衡量模型总的计算量，只是靠参数数量来评判训练时间其实有一些不太靠谱；而且参数众多可能意味着模型过度复杂，无效参数数量多，过拟合程度大。

其次，我像谈一下神经网络深度对训练的影响。过深的神经网络可以帮助模型捕捉更全面的特征信息，但是随之而来也会带来梯度爆炸/梯度消失问题。ResNet 中利用残差块结构很好地解决了这一点，受限于笔者的实验环境，我没有体验一下 50 层以上的神经网络之效果。但是像 MobileNet 这样看起来很深的架构效果却是 6 种架构种实际表现最差的——这主要是因为计算时把深度卷积和逐点卷积分开来算了，它实际还是一个像 LeNet 那样的轻量级架构。

在查阅 CNN 的相关论文、感受不同架构的思想同时，我也能感受到 CNN 的一个大的趋势——从大卷积核“粗提取”到小卷积核“细提取”，较新的几款架构已经很难见到大卷积核了，这样保证了对更复杂特征的抽取，同时也能增大神经网络的深度。

本次实验我们针对的是传统的 MNIST 数据集，它还是比较简单的，所有的模型在测试集上准确率均高于 97%，都是可以的。但是类似于 GoogLeNet, VGGNet 这样的架构并没有展现出特别明显的优势，LeNet 这样的简单架构反而还有着不错的效果。我在本次实验中并没有考虑使用数据增强技术，因为感觉针对 MNIST 数据集再去旋转/偏移/增添曝光度等方法意义不大；查阅了资料后我发现 CIFAR-10 可能更加适合做这个实验——60000 张 32×32 的彩色图片，受限于时间我也没有尝试。而且还有个值得考虑的因素是除了 LeNet 之外的其他架构并不是基于 MNIST 提出的，模型的泛化能力可能在此也受到了一定制约。

当然，我自我感觉自己设置的 epoch 太高了。我的 AlexNet, ResNet, VGGNet 都有明显的随着 epoch 增大在验证集上准确率先增大后减少的情况。这个也很好理解，因为 epoch 越多，模型过拟合的趋势就越明显，泛化能力越差。经过实测，最好的 epoch 轮次应该在 5 左右。

AlexNet 深度不算很深，训练时间也不算很长，但是效果却是 6 个架构中第二好的，看来 dropout 的功夫还是很强的，同样在 VGGNet 中，虽然我限制了它的输入尺寸，它的深度也不算很深，但是依然取得了不错的结果。

最后我还想感叹一句，自己最开始担心网络问题选择下载数据集并自行 load，放缩在实验快结束时看起来给自己增添了很多的烦恼，我严重怀疑自己的架构跑得那么慢主要是因为数据处理的不当，不知道哪个地方又多了些莫名其妙的多余运算。

总结和感想

通过本次实验，我对 CNN 的理解更进一步了。概括来说，我熟知了 CNN 中的几个重要组件——卷积层（获取局部信息）；池化层（提取重要特征、降维）；激活函数（非线性函数来拟合复杂的特性）；全连接层（将特征汇聚成结果）。我也针对几种架构的特点明确了 CNN 设计时的一些注意事项，虽然当前针对的仅仅只是简单的 MNIST 数据集分类任务，但相信这位日后学习 CV 相关的技术打下了坚实的基础。